

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

Course Name	Computer Vision with OpenCV
Course Code	DSA02
Course Outcome	CO5: Understand video processing – motion computation – 3D vision – geometry (BTL 6)
Name and Reg No	V M Sai Bhargav - 192324126
Questions Answered	Q1, Q3, and Q4

Question 1 (10 Marks): Real-Time Face Detection and Recognition Pipeline

Problem Understanding

Inputs: continuous live video frames from a camera. Outputs: each frame displayed with bounding boxes and identity labels for detected/recognized faces. Constraints: must run continuously without stopping, minimize per-frame delay, and correctly handle multiple faces in the same frame.

Pseudocode

```
BEGIN FaceDetectionRecognitionPipeline

// --- Module 1: Setup (runs once) ---
FUNCTION Initialize():
    faceDetector <- LOAD_HAAR_OR_DNN_FACE_DETECTOR()
    recognizer <- LOAD_TRAINED_RECOGNIZER(datasetPath)
    camera <- OPEN_CAMERA_STREAM()
    RETURN faceDetector, recognizer, camera

// --- Module 2: Preprocessing ---
FUNCTION Preprocess(frame):
    gray <- CONVERT_TO_GRAYSCALE(frame)
    gray <- APPLY_CLAHE(gray)    // normalizes lighting
    RETURN gray

// --- Module 3: Detection ---
FUNCTION DetectFaces(gray, faceDetector):
    faceList <- faceDetector.DETECT_MULTI_SCALE(gray)
    RETURN faceList    // list of (x, y, w, h) boxes

// --- Module 4: Recognition ---
FUNCTION RecognizeFace(gray, box, recognizer):
    faceROI <- CROP(gray, box)
    label, confidence <- recognizer.PREDICT(faceROI)
    IF confidence < THRESHOLD THEN
        RETURN label
    ELSE
```

```

        RETURN "Unknown"
    END IF

    // --- Module 5: Main real-time loop ---
    faceDetector, recognizer, camera <- Initialize()

    WHILE camera IS OPEN DO
        frame <- camera.READ_FRAME()
        IF frame IS EMPTY THEN
            BREAK
        END IF

        gray <- Preprocess(frame)
        faceList <- DetectFaces(gray, faceDetector)

        FOR EACH box IN faceList DO    // handles multiple faces
            label <- RecognizeFace(gray, box, recognizer)
            DRAW_RECTANGLE(frame, box)
            DRAW_LABEL(frame, box, label)
        END FOR

        DISPLAY(frame)
        IF USER_PRESSES('q') THEN
            BREAK
        END IF
    END WHILE

    camera.RELEASE()
    CLOSE_ALL_DISPLAY_WINDOWS()
END FaceDetectionRecognitionPipeline

```

Justification

The pipeline is split into independent functions (Initialize, Preprocess, DetectFaces, RecognizeFace) so each stage can be modified or optimized without affecting the others. The single outer WHILE loop guarantees continuous operation until the stream ends or the user exits, satisfying the 'operate continuously' constraint. Grayscale conversion and CLAHE keep per-frame preprocessing cheap, minimizing delay, while the inner FOR EACH loop over detected boxes explicitly handles any number of simultaneous faces without restructuring the pipeline.

Question 3 (10 Marks): Motion Detection and Tracking System

Problem Understanding

Inputs: continuous video frames from a surveillance camera. Outputs: video display with highlighted motion regions and tracked object identities. Constraints: the background may change dynamically (lighting, foliage) and the system must run in near real-time.

Pseudocode

```

BEGIN MotionDetectionTrackingSystem

```

```

FUNCTION Initialize():
    video <- OPEN_VIDEO_STREAM()
    bgSubtractor <- CREATE_BACKGROUND_SUBTRACTOR_MOG2()
    trackedObjects <- EMPTY_LIST    // stores {id, centroid}
    nextID <- 0
    RETURN video, bgSubtractor, trackedObjects, nextID

FUNCTION ExtractMotionMask(frame, bgSubtractor):
    blurred <- GAUSSIAN_BLUR(frame)
    fgMask <- bgSubtractor.APPLY(blurred)    // adapts to dynamic background
    fgMask <- MORPHOLOGICAL_OPEN(fgMask)
    fgMask <- MORPHOLOGICAL_CLOSE(fgMask)
    RETURN fgMask

FUNCTION FindMotionRegions(fgMask, minArea):
    contours <- FIND_CONTOURS(fgMask)
    regions <- EMPTY_LIST
    FOR EACH c IN contours DO
        IF AREA(c) >= minArea THEN    // filters residual noise
            APPEND regions, BOUNDING_BOX(c)
        END IF
    END FOR
    RETURN regions

FUNCTION UpdateTracks(trackedObjects, regions, nextID):
    newTracks <- EMPTY_LIST
    FOR EACH region IN regions DO
        centroid <- CENTROID(region)
        match <- FIND_NEAREST(trackedObjects, centroid, maxDistance)
        IF match EXISTS THEN
            APPEND newTracks, {id: match.id, centroid: centroid}
        ELSE
            APPEND newTracks, {id: nextID, centroid: centroid}
            nextID <- nextID + 1
        END IF
    END FOR
    RETURN newTracks, nextID

// --- Main loop ---
video, bgSubtractor, trackedObjects, nextID <- Initialize()

WHILE video HAS NEXT FRAME DO
    frame <- video.READ_FRAME()
    fgMask <- ExtractMotionMask(frame, bgSubtractor)
    regions <- FindMotionRegions(fgMask, minArea = 500)
    trackedObjects, nextID <- UpdateTracks(trackedObjects, regions, nextID)

    FOR EACH track IN trackedObjects DO
        DRAW_RECTANGLE(frame, track)
        DRAW_ID_LABEL(frame, track.id, track.centroid)
    END FOR

    DISPLAY(frame)

```

```

        IF USER_PRESSES('q') THEN
            BREAK
        END IF
    END WHILE

    video.RELEASE()
END MotionDetectionTrackingSystem

```

Justification

MOG2 background subtraction continuously updates its background model, so it correctly separates motion from a dynamic background instead of falsely flagging gradual scene changes as motion. Morphological opening/closing and the minimum-area filter in FindMotionRegions remove small noise blobs before they reach the tracker. Centroid-based nearest-neighbour matching in UpdateTracks is an $O(n)$ operation per frame, keeping the main loop lightweight enough to satisfy the near real-time constraint while still preserving object identity across frames.

Question 4 (10 Marks): Vehicle Counting System using Video

Problem Understanding

Inputs: video frames from a traffic camera. Outputs: a running vehicle count displayed on-screen.

Constraints: the system must avoid counting the same vehicle more than once and must correctly handle several vehicles present in the frame at the same time.

Pseudocode

```

BEGIN VehicleCountingSystem

    FUNCTION Initialize():
        video <- OPEN_VIDEO_STREAM()
        bgSubtractor <- CREATE_BACKGROUND_SUBTRACTOR_MOG2()
        countingLineY <- DEFINE_LINE_POSITION()
        trackedVehicles <- EMPTY_LIST // {id, centroid, counted}
        nextID <- 0
        vehicleCount <- 0
        RETURN video, bgSubtractor, countingLineY, trackedVehicles, nextID,
        vehicleCount

    FUNCTION DetectVehicles(frame, bgSubtractor, roi):
        fgMask <- bgSubtractor.APPLY(frame)
        fgMask <- MORPHOLOGICAL_OPEN(fgMask)
        contours <- FIND_CONTOURS(fgMask)
        boxes <- EMPTY_LIST
        FOR EACH c IN contours DO
            IF AREA(c) >= minVehicleArea AND INSIDE(c, roi) THEN
                APPEND boxes, BOUNDING_BOX(c)
            END IF
        END FOR
        RETURN boxes

    FUNCTION UpdateAndCount(trackedVehicles, boxes, nextID, vehicleCount, lineY):

```

```

updated <- EMPTY_LIST
FOR EACH box IN boxes DO    // handles multiple vehicles
  centroid <- CENTROID(box)
  match <- FIND_NEAREST(trackedVehicles, centroid, maxDistance)
  IF match EXISTS THEN
    prevY <- match.centroid.y
    IF prevY < lineY AND centroid.y >= lineY AND NOT match.counted THEN
      vehicleCount <- vehicleCount + 1    // crosses line exactly once
      match.counted <- TRUE
    END IF
    APPEND updated, {id: match.id, centroid: centroid, counted:
      match.counted}
  ELSE
    APPEND updated, {id: nextID, centroid: centroid, counted: FALSE}
    nextID <- nextID + 1
  END IF
END FOR
RETURN updated, nextID, vehicleCount

// --- Main loop ---
video, bgSubtractor, lineY, trackedVehicles, nextID, vehicleCount <-
Initialize()

WHILE video HAS NEXT FRAME DO
  frame <- video.READ_FRAME()
  boxes <- DetectVehicles(frame, bgSubtractor, roi)
  trackedVehicles, nextID, vehicleCount <-
    UpdateAndCount(trackedVehicles, boxes, nextID, vehicleCount, lineY)

  DRAW_LINE(frame, lineY)
  FOR EACH v IN trackedVehicles DO
    DRAW_RECTANGLE(frame, v)
  END FOR
  DRAW_TEXT(frame, "Count: " + vehicleCount)

  DISPLAY(frame)
  IF USER_PRESSES('q') THEN
    BREAK
  END IF
END WHILE

video.RELEASE()
END VehicleCountingSystem

```

Justification

Each vehicle is assigned a persistent ID via nearest-centroid matching, and a 'counted' flag on that ID ensures the count only increments the first time its centroid crosses the counting line, which directly prevents double counting. Because detection, matching, and counting are performed independently for every box in the FOR EACH loop, the design naturally scales to several vehicles present in the same frame without any change to the control logic.

Declaration: *I certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*